

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN MÔN KỸ THUẬT LẬP TRÌNH

Đề tài 2: Xây dựng ứng dụng quản lý bán hàng quần áo thời trang

Giảng viên hướng dẫn: Ts Phạm Đình Phong

Nhóm thực hiện : Nhóm 2

Sinh viên thực hiện:

STT	HỌ VÀ TÊN	MÃ SV	GHI CHÚ
6	Nguyễn Hồng An	253630650	
7	Vũ Mai Anh	253610648	
8	Nguyễn Lê Bảo Châu	253630650	
9	Trương Thị Minh Châu	253630651	
10	Nguyễn Thành Công	253630652	
11	Phạm Phú Cường	253630653	

MỤC LỤC

I. QUẢN LÝ SẢN PHẨM	8
1. Mục tiêu và ý nghĩa	8
2. Cấu trúc dữ liệu	8
2.1. Giải thích các thuộc tính (Attributes)	9
2.2. Cơ chế quản lý danh sách	9
2.3. Cấu trúc dữ liệu lưu trữ	9
3. Phân tích các hàm Chức năng trọng tâm	10
3.1. Thêm sản phẩm.....	10
3.1.1. Mục đích.....	10
3.1.2. Các điểm kỹ thuật trọng tâm	10
3.1.3. Quy trình xử lý (Workflow)	11
3.1.4. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)	11
3.2. Hiện thị sản phẩm	13
3.2.1. Mục đích.....	13
3.2.2. Kỹ thuật lập trình trọng tâm	13
3.2.3. Quy trình xử lý (Workflow)	13
3.3. Tìm theo mã.....	14
3.3.1. Mục đích.....	14
3.3.2. Các điểm kỹ thuật trọng tâm	14
3.3.3. Quy trình xử lý (Workflow)	15
3.4.1. Mục đích.....	16
3.4.2. Các điểm kỹ thuật trọng tâm	16
3.4.3. Quy trình xử lý (Workflow)	16
3.5. Sửa sản phẩm.....	17
3.5.1. Mục đích.....	17
3.5.2. Các điểm kỹ thuật trọng tâm	18
3.5.3. Quy trình xử lý (Workflow)	18
3.6. Xóa sản phẩm	20
3.6.1. Mục đích.....	20
3.6.2. Các điểm kỹ thuật trọng tâm	20

3.6.3. Quy trình xử lý (Workflow)	20
II. QUẢN LÝ KHO	22
1. Mục tiêu và ý nghĩa	22
2. Cấu trúc dữ liệu	22
3. Phân tích các hàm chức năng trọng tâm	23
3.1. Hàm nhập kho (nhapkho)	23
3.1.1. Mục đích.....	23
3.1.2. Các điểm kỹ thuật trọng tâm	23
3.1.3. Quy trình xử lý (Workflow)	24
3.1.4. Bảng minh họa sản phẩm mẫu	24
3.2. Hàm kiểm tra tồn kho (kiemtratonkho)	24
Hình 3.2 :Giao diện thực thi chức năng Kiểm tra tồn kho trên màn hình Console.....	25
3.2.1. Mục đích.....	25
3.2.2. Kỹ thuật lập trình trọng tâm	25
3.2.3. Bảng minh họa sản phẩm mẫu	25
3.3. Hàm cảnh báo hết hàng (canhbaohethang)	26
Hình 3.3:Giao diện thực thi chức năng Cảnh báo hết hàng trên màn hình Console.....	26
3.3.1. Mục đích.....	26
3.3.2. Kỹ thuật lập trình trọng tâm	26
3.3.3. Bảng minh họa sản phẩm mẫu	27
3.4. Hàm thống kê tổng tồn kho (thongketongtonkho)	27
Hình 3.4: Giao diện thực thi chức năng Thống kê tổng tồn kho trên màn hình Console.....	27
3.4.1. Mục đích.....	27
3.4.2. Kỹ thuật lập trình trọng tâm	27
3.4.3. . Bảng minh họa sản phẩm mẫu	28
3.5. Hàm tìm sản phẩm tồn kho nhiều nhất / ít nhất (tonkhonhieunhat / tonkhoitnhat).....	28
Hình 3.5.1:Giao diện thực thi chức năng tìm sản phẩm tồn kho nhiều nhất trên màn hình Console.....	28
Hình 3.5.2:Giao diện thực thi chức năng tìm sản phẩm tồn kho ít nhất trên màn hình Console.....	29

3.5.1. Mục đích.....	29
Giúp người quản lý định vị nhanh chóng hai thái cực của hàng hóa: Mặt hàng nào đang tồn đọng quá nhiều (cần xả kho, kích cầu) và mặt hàng nào đang ít nhất (cần ưu tiên tái đặt hàng).	29
3.5.2. Kỹ thuật lập trình trọng tâm (Thuật toán tìm cực trị)	29
3.5.3. Bảng minh họa sản phẩm mẫu	29
3.6. Hàm điều hướng Menu Kho (menukho)	30
Hình 3.6:Giao diện thực thi chức năng điều hướng Menu Kho trên màn hình Console.....	30
3.6.1. Mục đích.....	30
3.6.2. Kỹ thuật lập trình trọng tâm	31
3.6.3. . Bảng minh họa sản phẩm mẫu	31
III.QUẢN LÝ BÁN HÀNG	32
1. Mục tiêu và ý nghĩa	32
2. Cấu trúc dữ liệu	32
2.1. Tóm tắt quy trình chức năng bán hàng.....	33
2.2 Cấu trúc hóa đơn.....	34
3. Phân tích các hàm chức năng trọng tâm	34
3.1 Hiển thị thông tin sản phẩm.....	34
3.2. Nhập và kiểm tra số lượng mua.....	34
3.3. Cập nhật dữ liệu kho hàng.....	35
3.4. Tạo hóa đơn bán hàng	35
3.5. Lưu hóa đơn vào hệ thống.....	36
3.6. In hóa đơn cho khách hàng.....	36
3.7. Thông báo hoàn tất giao dịch	36
3. 8. Chạy thử phần mềm.....	37
3.8.1 Các bước chạy thử.....	37
Bước 1: Từ menu chính chọn	37
Bước 2: Chọn chức năng Bán hàng.....	37
Bước 3: Nhập mã sản phẩm	38
Bước 4: Chương trình hiển thị thông tin sản phẩm.....	38
Bước 5: Nhập số lượng khách mua	38
Bước 6: Hệ thống xử lý	38
Bước 7: Hóa đơn được in ra	39

Bước 8: Thông báo thành công	39
Kiểm tra hóa đơn vừa tạo	39
Kiểm tra doanh thu	40
Chạy thử hoàn chỉnh.....	40
3.8.2 Mục đích:.....	40
IV. THỐNG KÊ	41
1. Mục tiêu và ý nghĩa	41
2. Cấu trúc dữ liệu	41
3. Phân tích các hàm chức năng trọng tâm.....	42
3.1. Hàm thống kê doanh thu.....	42
3.1.1. Mục đích.....	42
3.1.2. Kỹ thuật lập trình trọng tâm	42
3.1.4. Bảng minh họa sản phẩm mẫu	43
3.2. Hàm thống kê lợi nhuận	43
3.2.1. Mục đích.....	43
3.2.2. Kỹ thuật lập trình trọng tâm	43
3.2.3. Quy trình xử lý	44
3.3. Hàm tìm sản phẩm bán chạy nhất	44
3.3.1. Mục đích.....	44
3.3.2. Kỹ thuật lập trình trọng tâm	45
3.3.3. Quy trình xử lý	45
3.4. Hàm tìm sản phẩm bán ít nhất.....	45
3.4.1. Mục đích.....	45
3.4.2. Kỹ thuật lập trình trọng tâm	46
3.4.3. Quy trình xử lý	46
3.5. Hàm sắp xếp theo giá bán.....	47
3.5.1. Mục đích.....	47
3.5.2. Kỹ thuật lập trình trọng tâm	48
3.5.3. Quy trình xử lý	48
3.5.4. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)	48
3.6. Hàm điều hướng Menu Thống kê.....	48
3.6.1. Mục đích.....	48
3.6.2. Kỹ thuật lập trình trọng tâm	49

3.6.3. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử).....	49
3.6.3. Quy trình xử lý (Workflow).....	50
4.Kết luận:.....	50

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển, việc ứng dụng các phần mềm quản lý vào hoạt động kinh doanh đã trở thành một nhu cầu thiết yếu. Đối với các cửa hàng thời trang, việc quản lý sản phẩm, theo dõi số lượng hàng hóa trong kho, thực hiện bán hàng và thống kê doanh thu bằng phương pháp thủ công thường gặp nhiều khó khăn, tốn thời gian và dễ xảy ra sai sót. Vì vậy, việc xây dựng một chương trình hỗ trợ quản lý là cần thiết nhằm nâng cao hiệu quả làm việc và giảm thiểu những hạn chế trong quá trình quản lý.

Xuất phát từ thực tế đó, nhóm chúng em đã lựa chọn chủ đề “Xây dựng ứng dụng quản lý bán hàng quần áo thời trang” trong khuôn khổ học phần môn Kỹ thuật lập trình. Chương trình được phát triển bằng ngôn ngữ C với giao diện đơn giản, thân thiện và dễ sử dụng. Hệ thống tập trung vào các chức năng cơ bản nhưng quan trọng của một cửa hàng thời trang bao gồm: quản lý sản phẩm, quản lý kho hàng, bán hàng và thống kê dữ liệu.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ vận dụng các kiến thức đã học như cấu trúc dữ liệu, hàm, mảng, tệp tin và xử lý dữ liệu mà còn rèn luyện kỹ năng phân tích yêu cầu, thiết kế thuật toán, xây dựng chương trình và làm việc nhóm. Đây là cơ hội để nhóm tiếp cận với một bài toán quản lý thực tế, từ đó nâng cao khả năng tư duy logic và kỹ năng lập trình.

Nội dung báo cáo sẽ trình bày quá trình phân tích bài toán, thiết kế chương trình, cài đặt các chức năng chính và đánh giá kết quả đạt được. Qua đó, nhóm mong muốn mang đến một sản phẩm có tính ứng dụng thực tiễn, đáp ứng được các yêu cầu cơ bản trong việc quản lý hoạt động của một cửa hàng thời trang.

I. QUẢN LÝ SẢN PHẨM

1. Mục tiêu và ý nghĩa

Module Quản lý sản phẩm là thành phần nền tảng của hệ thống, cho phép người dùng quản trị toàn bộ danh mục hàng hóa. Chức năng không chỉ dừng lại ở việc lưu trữ mà còn đảm bảo dữ liệu luôn được chuẩn hóa, nhất quán và an toàn thông qua cơ chế quản lý tệp tin.

2. Cấu trúc dữ liệu

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FILE_SP "sanpham.txt"
#define FILE_HD "hoadon.txt"

typedef struct{
    int masp;
    char tensp[50];
    char loai[30];
    char size[20];
    char mausac[20];
    float gianhap;
    float giaban;
    int soluong;
    int daban;
} sanpham;

typedef struct node{
    sanpham data;
    struct node *next;
} node;
```

Hình 1.1: Khai báo cấu trúc dữ liệu sản phẩm và danh sách liên kết trong chương trình

Hệ thống áp dụng cấu trúc Danh sách liên kết đơn (Linked List) để quản lý dữ liệu.

- Cấu trúc (struct): Chương trình sử dụng struct sanpham để định nghĩa 9 thuộc tính sản phẩm và struct node để đóng gói dữ liệu vào danh sách. Việc sử dụng danh sách liên kết giúp hệ thống xử lý số lượng sản phẩm linh hoạt, không bị giới hạn bộ nhớ như mảng tĩnh.
- Cơ chế lưu trữ: Mọi thay đổi đều được đồng bộ hóa với tệp sanpham.txt, giúp dữ liệu tồn tại vĩnh viễn sau khi chương trình đóng.

2.1. Giải thích các thuộc tính (Attributes)

- **Mã sản phẩm (masp):** Đóng vai trò là khóa chính (Primary Key) để định danh duy nhất cho từng sản phẩm, giúp việc tìm kiếm, sửa và xóa trở nên chính xác.
- **Thông tin mô tả (tensp, loại, size, mausac):** Các trường chuỗi được khai báo với độ dài cố định để đảm bảo tính ổn định khi đọc/ghi file.
- **Dữ liệu kinh doanh (gianhap, giaban, soluong, daban):** Các trường kiểu số (float, int) hỗ trợ các phép toán thống kê như tính doanh thu, lợi nhuận và kiểm tra tồn kho.

2.2. Cơ chế quản lý danh sách

- **Con trỏ head:** Luôn trỏ vào phần tử đầu tiên của danh sách, là điểm truy cập duy nhất để bắt đầu mọi thao tác duyệt (tìm kiếm, hiển thị, tính toán).
- **Tính liên kết (Pointers):** Mỗi nút chứa một con trỏ *next trỏ đến địa chỉ của nút tiếp theo. Khi thực hiện xóa, chương trình sẽ thay đổi liên kết này để "cắt bỏ" nút cũ mà không làm gián đoạn toàn bộ danh sách.

2.3. Cấu trúc dữ liệu lưu trữ

```
masp tensp loại size màu gianhap giaban soluong
101 Aothun basic Aothun M Đen 80000.00 150000.00 50
102 Áo crop top Aothun S Trắng 90000.00 170000.00 40
201 Sômi slimfit Aosomi L Xanh 200000.00 350000.00 30
202 Sômi oxford Aosomi XL Xanh 220000.00 380000.00 25
301 Quanjian skinny Quanjian M Đen 300000.00 550000.00 20
302 Quankaki classic Quankaki L Nâu 250000.00 450000.00 30
401 Mũ lưỡi trai Phukien Freesize Đen 50000.00 120000.00 100
402 Túi xách Phukien S Trắng 10000.00 15000.00 200
```

Hình 1.2: Cấu trúc dữ liệu trong tệp sanpham.txt

Chương trình sử dụng tệp văn bản (sanpham.txt) để lưu trữ dữ liệu bền vững. Cấu trúc tệp được thiết kế như sau:

- Dòng đầu tiên: Chứa tiêu đề các cột (masp, tensp, loại...) nhằm mục đích kiểm tra và định dạng dữ liệu trực quan cho người quản trị khi mở tệp.
- Các dòng tiếp theo: Mỗi dòng đại diện cho một sản phẩm, các thuộc tính được phân cách bởi khoảng trắng, giúp hàm fscanf trong C dễ dàng đọc và gán giá trị vào danh sách liên kết trong bộ nhớ RAM.

3. Phân tích các hàm Chức năng trọng tâm

3.1. Thêm sản phẩm

```
void themsanpham(){
    sanpham sp;
    printf("\n===== THEM SAN PHAM =====\n");
    printf("Ma san pham: ");
    scanf("%d",&sp.masp);
    if(timtheoma(sp.masp) != NULL){
        printf("Ma san pham da ton tai!\n");
        return;
    }
    getchar();
    printf("Ten san pham: ");
    fgets(sp.tensp,sizeof(sp.tensp),stdin);
    sp.tensp[strcspn(sp.tensp, "\n")] = '\0';
    printf("Loai san pham: ");
    fgets(sp.loai,sizeof(sp.loai),stdin);
    sp.loai[strcspn(sp.loai, "\n")] = '\0';
    printf("Size: ");
    fgets(sp.size,sizeof(sp.size),stdin);
    sp.size[strcspn(sp.size, "\n")] = '\0';
    printf("Mau sac: ");
    fgets(sp.mausac,sizeof(sp.mausac),stdin);
    sp.mausac[strcspn(sp.mausac, "\n")] = '\0';
    printf("Gia nhap: ");
    scanf("%f",&sp.gianhap);
    printf("Gia ban: ");
    scanf("%f",&sp.giaban);
    printf("So luong: ");
    scanf("%d",&sp.soluong);
    sp.daban = 0;
    themcuoi(sp);
    luusanpham();
    printf("\nThem thanh cong!\n");
}
```

Hình 2.1: Giao diện thực thi chức năng Thêm sản phẩm trên màn hình Console

3.1.1. Mục đích

Chức năng này cho phép người dùng đưa các mặt hàng mới vào danh mục quản lý của cửa hàng. Đây là bước khởi đầu quan trọng để hệ thống có dữ liệu phục vụ cho các module Bán hàng và Thống kê.

3.1.2. Các điểm kỹ thuật trọng tâm

- **Kiểm tra tính duy nhất (Unique Constraint):** Sử dụng hàm `timtheoma()` để rà soát mã sản phẩm trước khi thêm. Nếu mã đã tồn tại, hệ thống chặn thao tác (return), ngăn chặn việc trùng lặp dữ liệu — điều kiện tiên quyết để quản lý sản phẩm chính xác.

- **Xử lý bộ đệm bàn phím (Buffer Management):** Lệnh `getchar()` được đặt sau `scanf()` để làm sạch bộ đệm. Đây là kỹ thuật thiết yếu để ngăn chặn lỗi "nhảy dòng" khi người dùng chuyển từ việc nhập số sang nhập chuỗi.
- **Chuẩn hóa dữ liệu đầu vào (String Sanitization):**
 - Sử dụng `fgets` thay cho `scanf` để cho phép nhập dữ liệu chứa khoảng trắng (ví dụ: tên sản phẩm dài).
 - Lệnh `sp.tensp[strcspn(sp.tensp, "\n")] = '\0';` thực hiện loại bỏ ký tự xuống dòng thừa. Kỹ thuật này giúp dữ liệu lưu vào file `sanpham.txt` có cấu trúc phẳng, tránh lỗi định dạng khi đọc lại dữ liệu trong các phiên làm việc sau.
- **Cơ chế lưu trữ đồng bộ (Synchronization):** Sau mỗi thao tác chèn vào danh sách liên kết (`themcuoi`), hàm ngay lập tức gọi `luusanpham()` để đồng bộ hóa dữ liệu từ RAM xuống đĩa cứng. Điều này đảm bảo an toàn thông tin tối đa cho người dùng.

3.1.3. Quy trình xử lý (Workflow)

Quy trình thêm sản phẩm được thiết kế theo 4 bước chặt chẽ:

1. **Nhập thông tin ban đầu:** Người dùng nhập Mã sản phẩm.
2. **Kiểm tra tính hợp lệ:** Hệ thống thực hiện kiểm tra mã sản phẩm hiện nhập có bị trùng lặp hay không thông qua hàm `timtheoma()`. Nếu mã đã tồn tại, hệ thống từ chối thêm để đảm bảo tính nhất quán của dữ liệu.
3. **Nhập thông tin chi tiết:** Người dùng nhập các trường dữ liệu còn lại (Tên, Loại, Size, Màu, Giá nhập, Giá bán, Số lượng).
4. **Lưu trữ và Đồng bộ:**
 - Tạo mới một node chứa dữ liệu (`taonode`).
 - Chèn node vào cuối danh sách liên kết (`themcuoi`).
 - Ghi toàn bộ danh sách vào tệp `sanpham.txt` để bảo toàn dữ liệu (`luusanpham`).

3.1.4. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)

Hình ảnh dưới đây minh họa luồng tương tác thực tế của người dùng với module Quản lý sản phẩm:

```

=====
QUAN LY SHOP THOI TRANG
=====
1. Quan ly san pham
2. Quan ly kho
3. Ban hang
4. Thong ke
0. Thoat
=====
Chon: 1

=====
QUAN LY SAN PHAM
=====
1. Them san pham
2. Hien thi san pham
3. Tim theo ma
4. Tim theo ten
5. Sua san pham
6. Xoa san pham
0. Quay lai
=====
Chon: 1

===== THEM SAN PHAM =====
Ma san pham: 101
Ten san pham: Aothun basic
Loai san pham: Aothun
Size: M
Mau sac: Den
Gia nhap: 80000
Gia ban: 150000
So luong: 50

Them thanh cong!

```

Hình 2.2: Minh họa luồng tương tác thực tế của người dùng với module Quản lý sản phẩm

Hệ thống sử dụng menu phân cấp, giúp người quản trị dễ dàng định hướng và thực hiện nhanh các thao tác. Cụ thể, khi người dùng thực hiện tác vụ 'Thêm sản phẩm', hệ thống sẽ điều hướng qua menu con và yêu cầu nhập liệu tuần tự các thuộc tính của hàng hóa, đảm bảo tính đầy đủ của dữ liệu trước khi lưu trữ vào danh sách liên kết ."

Sau khi hoàn tất quá trình nhập liệu, toàn bộ danh mục sản phẩm được hệ thống tổng hợp và hiển thị dưới dạng bảng để người quản lý dễ dàng đối chiếu:

Ma	Ten SP	Loai	Size	Mau	GiaNhap	GiaBan	Ton	DaBan
101	Aothun basic	Aothun	M	Den	80000	150000	50	0
102	Ao croptop	Aothun	S	Trang	90000	170000	40	0
201	Somi slimfit	Aosomi	L	Xanh	200000	350000	30	0
202	Somi oxford	Aosomi	XL	Xam	220000	380000	25	0
301	Quanjean skinny	Quanjean	M	Den	300000	550000	20	0
302	Quankaki classic	Quankaki	L	Nau	250000	450000	30	0
401	Muluoitrai	Phukien	Frees	Den	50000	120000	100	0
402	Tatcongan	Phukien	S	Trang	10000	15000	200	0

Hình 2.3: Giao diện bảng danh sách sản phẩm sau khi đã lưu trữ thành công

3.2. Hiện thị sản phẩm

```
void hienthisanpham(sanpham sp){
    printf("| %-5d ", sp.masp);
    printf("| %-20.20s ", sp.tensp);
    printf("| %-10.10s ", sp.loai);
    printf("| %-5.5s ", sp.size);
    printf("| %-8.8s ", sp.mausac);
    printf("| %-10.0f ", sp.gianhap);
    printf("| %-10.0f ", sp.giaban);
    printf("| %-5d ", sp.soluong);
    printf("| %-5d |\n", sp.daban);
}

void hienthisanpham(){
    if(head == NULL){
        printf("\nDanh sach rong!\n");
        return;
    }
    printf("\n===== \n");
    printf("| %-5s | %-20s | %-10s | %-5s | %-8s | %-10s | %-10s | %-5s | %-5s |\n",
           "Ma", "Ten SP", "Loai", "Size", "Mau", "GiaNhap", "GiaBan", "Ton", "DaBan");
    printf("===== \n");
    node *p = head;
    while(p != NULL){
        hienthisanpham(p->data);
        p = p->next;
    }
}
```

Hình 3: Chi tiết logic hàm thêm sản phẩm, bao gồm cơ chế kiểm tra mã và xử lý chuỗi

3.2.1. Mục đích

Chức năng này giúp người quản trị có cái nhìn tổng quát về toàn bộ kho hàng. Thông qua bảng dữ liệu, người dùng có thể nhanh chóng nắm bắt các thông tin quan trọng như: tình trạng tồn kho, giá cả và số lượng hàng đã bán của từng mặt hàng.

3.2.2. Kỹ thuật lập trình trọng tâm

- **Duyệt danh sách liên kết:** Chương trình sử dụng vòng lặp while(p != NULL) để di chuyển qua từng mắt xích của danh sách. Đây là phương pháp truy cập dữ liệu tuyến tính cơ bản nhưng hiệu quả, đảm bảo mọi sản phẩm đều được hiển thị đầy đủ.
- **Định dạng bảng biểu (Tabular Formatting):** Sử dụng kỹ thuật căn lề trong printf:
 - Ví dụ: printf("| %-5d |", sp.masp);
 - Ký tự - trong %-5d có nghĩa là căn lề trái, con số 5 quy định độ rộng cột. Kỹ thuật này giúp bảng dữ liệu luôn thẳng hàng bất kể tên sản phẩm dài hay ngắn, tạo ra giao diện chuyên nghiệp như một phần mềm quản lý thực thụ.
- **Xử lý ngoại lệ:** Trước khi in, chương trình luôn kiểm tra if(head == NULL). Đây là bước kiểm tra an toàn, giúp thông báo rõ ràng cho người dùng khi kho hàng chưa có dữ liệu, thay vì để màn hình trống trải hoặc gây lỗi chương trình.

3.2.3. Quy trình xử lý (Workflow)

1. **Kiểm tra trạng thái:** Hệ thống kiểm tra biến head. Nếu là NULL, thông báo "Danh sách rỗng" và kết thúc hàm.
2. **Khởi tạo giao diện:** In dòng tiêu đề bảng với các cột rõ ràng (Mã, Tên, Loại, v.v.).
3. **Duyệt và xuất:**
 - o Khởi tạo con trỏ tạm p = head.
 - o Vòng lặp: In thông tin tại p->data -> Chuyển con trỏ p = p->next.
4. **Kết thúc:** Dừng lại khi p chạm NULL.

3.3. Tìm theo mã

```
void timkiemtheoma(){
    int ma;
    printf("\nNhap ma can tim: ");
    scanf("%d",&ma);
    node *p = timtheoma(ma);
    if(p == NULL){
        printf("Khong tim thay!\n");
        return;
    }
    printf("\nTim thay:\n");
    printf("Ma SP      : %d\n",p->data.masp);
    printf("Ten SP       : %s\n",p->data.tensp);
    printf("Loai        : %s\n",p->data.loai);
    printf("Size        : %s\n",p->data.size);
    printf("Mau sac     : %s\n",p->data.mausac);
    printf("Gia nhap    : %.0f\n",p->data.gianhap);
    printf("Gia ban     : %.0f\n",p->data.giaban);
    printf("So luong    : %d\n",p->data.soluong);
}
```

Hình 4.1: Giao diện hiển thị chi tiết thông tin sản phẩm sau khi tìm kiếm thành công theo mã"

3.3.1. Mục đích

Chức năng này giúp người dùng xác định thông tin của một sản phẩm cụ thể bằng cách sử dụng **Mã sản phẩm** làm khóa định danh duy nhất. Đây là chức năng được gọi thường xuyên trong các thao tác nghiệp vụ như sửa thông tin hoặc xóa sản phẩm.

3.3.2. Các điểm kỹ thuật trọng tâm

- **Duyệt tuần tự (Linear Search):** Chương trình sử dụng vòng lặp while để duyệt qua từng node trong danh sách liên kết. Với mỗi node,

hệ thống so sánh mã sản phẩm hiện tại (p->data.masp) với mã cần tìm (ma).

- **Tính tối ưu của con trỏ:** Điểm mạnh của hàm này là nó trả về trực tiếp một con trỏ kiểu node *. Điều này cho phép các hàm khác (như suasanpham hoặc xoasanpham) có thể sử dụng ngay địa chỉ của sản phẩm đó mà không cần phải thực hiện tìm kiếm lại, giúp tối ưu hóa hiệu năng hệ thống.
- **Xử lý tình huống không tìm thấy:** Nếu vòng lặp kết thúc mà không tìm thấy khớp mã, hàm sẽ trả về NULL. Hệ thống sau đó dựa vào giá trị này để thông báo cho người dùng, tránh tình trạng ứng dụng bị treo hoặc báo lỗi không xác định.

3.3.3. Quy trình xử lý (Workflow)

1. **Tiếp nhận yêu cầu:** Người dùng nhập mã sản phẩm cần tra cứu thông qua bàn phím.
2. **Khởi tạo tìm kiếm:** Gán con trỏ tạm p = head để bắt đầu từ phần tử đầu tiên.
3. **So sánh:**
 - Nếu p->data.masp == ma: Trả về con trỏ p và kết thúc hàm.
 - Nếu không khớp: Chuyển sang phần tử kế tiếp (p = p->next).
4. **Kết thúc:** Nếu p trở thành NULL, nghĩa là đã duyệt hết danh sách mà không tìm thấy mã, thông báo "Khong tim thay!".

```
=====
                QUAN LY SAN PHAM
=====
1. Them san pham
2. Hien thi san pham
3. Tim theo ma
4. Tim theo ten
5. Sua san pham
6. Xoa san pham
0. Quay lai
=====
Chon: 3

Nhap ma can tim: 102

Tim thay:
Ma SP      : 102
Ten SP     : Ao croptop
Loai       : Aothun
Size       : S
Mau sac    : Trang
Gia nhap   : 90000
Gia ban    : 170000
So luong   : 40
```

Hình 4.2: Giao diện thực thi chức năng tìm kiếm sản phẩm theo mã

3.4. Tìm theo tên

```

void timkiemtheoten(){
    char ten[50];
    getchar();
    printf("\nNhap ten can tim: ");
    fgets(ten,sizeof(ten),stdin);
    ten[strlen(ten)-1] = '\0';
    int found = 0;
    node *p = head;
    while(p != NULL){
        if(strstr(p->data.tensp,ten) != NULL){
            if(found == 0){
                printf("\nKet qua tim kiem:\n");
                printf("=====\\n");
                printf("%-8s%-25s%-15s%-10s%-12s%-12s%-10s%-10s\\n", "Ma", "Ten SP", "Loai", "Size", "Mau", "GiaNhap", "GiaBan", "Ton", "DaBan");
                printf("=====\\n");
            }
            hienthisanpham(p->data);
            found = 1;
        }
        p = p->next;
    }
    if(found == 0){
        printf("Khong tim thay!\\n");
    }
}

```

Hình 5.1: Mã nguồn chi tiết chức năng tìm kiếm sản phẩm theo tên

3.4.1. Mục đích

Chức năng này hỗ trợ người dùng tra cứu sản phẩm dựa trên từ khóa là tên sản phẩm. Đây là một tính năng **tối ưu hóa trải nghiệm người dùng (UX)**, cho phép tìm kiếm linh hoạt thay vì phải nhập chính xác mã sản phẩm.

3.4.2. Các điểm kỹ thuật trọng tâm

- **Tìm kiếm gần đúng (Partial Match Search):** Hệ thống sử dụng hàm strstr(chuoi_goc, chuoi_tim) từ thư viện <string.h>. Kỹ thuật này giúp tìm kiếm các sản phẩm có tên chứa từ khóa mà người dùng nhập vào. Ví dụ: khi nhập "Ao", hệ thống sẽ hiển thị toàn bộ các sản phẩm bắt đầu bằng "Ao" như "Ao thun", "Ao khoác", "Ao so mi".
- **Xử lý danh sách liên kết:** Chương trình duyệt tuần tự từ head đến NULL. Tại mỗi nút (node), hệ thống kiểm tra điều kiện chứa từ khóa. Nếu khớp, sản phẩm đó sẽ được in ra bảng.
- **Cơ chế đếm kết quả:** Biến found được sử dụng để kiểm soát giao diện hiển thị. Nếu không tìm thấy sản phẩm nào (found == 0), hệ thống sẽ thông báo ngay lập tức, đảm bảo luồng phản hồi luôn minh bạch.

3.4.3. Quy trình xử lý (Workflow)

1. **Nhập từ khóa:** Người dùng nhập tên hoặc một phần tên sản phẩm.
2. **Duyệt danh sách:** Hệ thống duyệt từ đầu đến cuối danh sách liên kết.
3. **So sánh:** Sử dụng strstr để kiểm tra sự tồn tại của từ khóa trong tensp của từng sản phẩm.

4. **Hiển thị:** Nếu khớp, gọi hàm `hienthi1sanpham` để in thông tin. Nếu duyệt hết danh sách mà biến `found` vẫn bằng 0, thông báo "Không tìm thấy!".

Sau khi nhập từ khóa tìm kiếm (ví dụ: 'Ao croptop'), hệ thống thực hiện quét qua danh sách liên kết, so sánh dữ liệu và trình bày kết quả đạt chuẩn định dạng bảng như minh họa dưới đây:

```
=====
      QUAN LY SAN PHAM
=====
1. Them san pham
2. Hien thi san pham
3. Tim theo ma
4. Tim theo ten
5. Sua san pham
6. Xoa san pham
0. Quay lai
=====
Chon: 4

Nhap ten can tim: Ao croptop

Ket qua tim kiem:
=====
Ma      Ten SP      Loai      Size      Mau      GiaNhap      GiaBan      Ton      DaBan
=====
| 102   | Ao croptop | Aothun   | S        | Trang   | 90000      | 170000     | 40     | 0     |
```

Hình 5.2: Giao diện kết quả sau khi thực hiện chức năng tìm kiếm sản phẩm theo tên

3.5. Sửa sản phẩm

```
void suasanpham(){
    int ma;
    printf("\nNhap ma can sua: ");
    scanf("%d",&ma);
    node *p = timtheoma(ma);
    if(p == NULL){
        printf("Không tìm thấy!\n");
        return;
    }
    getchar();
    printf("Ten moi: ");
    fgets(p->data.tensp, sizeof(p->data.tensp), stdin);
    printf("Loai moi: ");
    fgets(p->data.loai, sizeof(p->data.loai), stdin);
    printf("Size moi: ");
    fgets(p->data.size, sizeof(p->data.size), stdin);
    printf("Mau moi: ");
    fgets(p->data.mausac, sizeof(p->data.mausac), stdin);
    printf("Gia nhap moi: ");
    scanf("%f",&p->data.gianhap);
    printf("Gia ban moi: ");
    scanf("%f",&p->data.giaban);
    printf("So luong moi: ");
    scanf("%d",&p->data.soluong);
    luusanpham();
    printf("\nCap nhat thanh cong!\n");
}
```

Hình 6.1: Mã nguồn chức năng cập nhật thông tin sản phẩm

3.5.1. Mục đích

Chức năng này cho phép người quản lý cập nhật thông tin sản phẩm khi có sự thay đổi thực tế về giá nhập, giá bán, số lượng hoặc các thông tin mô tả

khác. Việc chỉnh sửa trực tiếp trên danh sách liên kết giúp dữ liệu luôn phản ánh đúng tình trạng hiện tại của cửa hàng.

3.5.2. Các điểm kỹ thuật trọng tâm

- **Xác thực trước khi sửa:** Chương trình sử dụng hàm `timkiemtheoma()` để định vị sản phẩm. Nếu không tìm thấy mã, hệ thống sẽ từ chối thao tác, ngăn chặn việc cập nhật dữ liệu rác.
- **Cập nhật dữ liệu tại chỗ (In-place Update):** Sau khi tìm được địa chỉ nút `p`, hệ thống yêu cầu nhập dữ liệu mới và ghi đè trực tiếp vào các trường (`p->data.gianhap`, `p->data.giaban`,...).
- **Đồng bộ hóa (Synchronization):** Điểm mấu chốt là hàm luôn gọi `luusanpham()` ngay sau khi sửa. Điều này đảm bảo rằng dữ liệu mới trên bộ nhớ RAM luôn được lưu lại trên tệp `sanpham.txt`, giúp dữ liệu không bị mất nếu có sự cố xảy ra.

3.5.3. Quy trình xử lý (Workflow)

1. **Tìm kiếm:** Người dùng nhập mã sản phẩm. Nếu sản phẩm không tồn tại, báo lỗi và thoát.
 2. **Nhập mới:** Nếu tìm thấy, người dùng nhập các thuộc tính mới (Tên, Giá, Số lượng...).
 3. **Cập nhật:** Dữ liệu mới được ghi đè vào nút tương ứng trong danh sách liên kết.
 4. **Đồng bộ:** Gọi hàm ghi tệp để lưu trữ thay đổi.
 5. **Thông báo:** Hệ thống báo "Sửa thành công" để người dùng xác nhận.
- Hình ảnh dưới đây minh họa luồng thực thi, từ việc định danh sản phẩm qua mã đến quá trình ghi đè dữ liệu mới vào hệ thống:

```

=====
                        QUAN LY SAN PHAM
=====
1. Them san pham
2. Hien thi san pham
3. Tim theo ma
4. Tim theo ten
5. Sua san pham
6. Xoa san pham
0. Quay lai
=====
Chon: 5

Nhap ma can sua: 401
Ten moi: Guong mini
Loai moi: Phukien
Size moi: Nho
Mau moi: Be
Gia nhap moi: 1500
Gia ban moi: 3000
So luong moi: 100

Cap nhap thanh cong!

```

Hình 6.2: Giao diện thực thi chức năng sửa thông tin sản phẩm

Kết quả hiển thị bảng danh sách sản phẩm sau khi sửa được như sau:

Ma	Ten SP	Loai	Size	Mau	GiaNhap	GiaBan	Ton	DaBan
1	Aothun basic	Aothun	M	Den	80000	150000	50	0
102	Ao croptop	Aothun	S	Trang	90000	170000	40	0
201	Somi slimfit	Aosomi	L	Xanh	200000	350000	30	0
202	Somi oxford	Aosomi	XL	Xam	220000	380000	25	0
301	Quanjean skinny	Quanjean	M	Den	300000	550000	20	0
302	Quankaki classic	Quankaki	L	Nau	250000	450000	30	0
401	Guong mini	Phukien	Nho	Be	1500	3000	100	0
402	Tatcongan	Phukien	S	Trang	10000	15000	200	0

Hình 6.3: Danh sách sản phẩm sau khi cập nhật thành công thông tin sản phẩm mã 401

3.6. Xóa sản phẩm

```
void xoasanpham(){
    int ma;
    printf("\nNhap ma can xoa: ");
    scanf("%d",&ma);
    node *p = head;
    node *truoc = NULL;
    while(p != NULL){
        if(p->data.masp == ma){
            if(truoc == NULL){
                head = p->next;
            }
            else{
                truoc->next = p->next;
            }
            free(p);
            luusanpham();
            printf("Xoa thanh cong!\n");
            return;
        }
        truoc = p;
        p = p->next;
    }
    printf("Khong tim thay san pham!\n");
}
```

Hình 7.1: Mã nguồn chi tiết chức năng xóa sản phẩm sử dụng kỹ thuật quản lý bộ nhớ (con trỏ kép)

3.6.1. Mục đích

Hàm này cho phép loại bỏ hoàn toàn một sản phẩm khỏi hệ thống khi sản phẩm đó ngừng kinh doanh hoặc bị xóa nhầm. Đây là thao tác quản trị trực tiếp trên danh sách liên kết đơn.

3.6.2. Các điểm kỹ thuật trọng tâm

- **Xử lý con trỏ:** Đây là phần khó nhất của danh sách liên kết. Đây là kỹ thuật chuẩn khi xóa một nút trong danh sách liên kết đơn. Con trỏ p dùng để duyệt và tìm kiếm, trong khi truoc luôn theo sát ngay phía sau để giúp bạn "nối" lại liên kết sau khi nút cần xóa bị loại bỏ.
- **Cơ chế hủy nút (Deallocation):**
 - Sau khi tìm thấy mã cần xóa, chương trình điều chỉnh lại liên kết: `truoc->next = p->next`.
 - Lệnh `free(p)` cực kỳ quan trọng để giải phóng bộ nhớ, tránh rò rỉ bộ nhớ (memory leak).
- **Trường hợp đặc biệt:** Chương trình xử lý riêng trường hợp xóa phần tử đầu tiên (head) bằng cách kiểm tra `if(truoc == NULL)`.

3.6.3. Quy trình xử lý (Workflow)

1. **Tìm kiếm:** Người dùng nhập mã. Nếu không tìm thấy, báo lỗi.
2. **Xác định vị trí:** Duyệt danh sách, con trỏ p đi tới nút cần xóa, con trỏ trước giữ nút đứng ngay trước p.
3. **Cập nhật liên kết:** Nối nút trước trực tiếp với p->next.
4. **Giải phóng:** Gọi free(p) để xóa bỏ dữ liệu khỏi bộ nhớ RAM.
5. **Đồng bộ:** Gọi luusanpham() để cập nhật file sanpham.txt sau khi đã xóa phần tử.

Sau khi thực hiện yêu cầu xóa sản phẩm, hệ thống sẽ thực hiện giải phóng bộ nhớ cho nút dữ liệu tương ứng và đồng bộ hóa lại file lưu trữ. Hình dưới đây minh họa kết quả thực thi chức năng xóa sản phẩm:

```

=====
                QUAN LY SAN PHAM
=====
1. Them san pham
2. Hien thi san pham
3. Tim theo ma
4. Tim theo ten
5. Sua san pham
6. Xoa san pham
0. Quay lai
=====
Chon: 6

Nhap ma can xoa: 401
Xoa thanh cong!

```

Hình 7.2: Giao diện thực thi chức năng xóa sản phẩm

Sau khi thực hiện lệnh xóa sản phẩm mã '401', hệ thống sẽ tự động cập nhật lại danh sách và loại bỏ hoàn toàn sản phẩm đó khỏi bộ nhớ. Kết quả kiểm tra danh sách sau khi xóa được minh họa như hình dưới đây:

Ma	Ten SP	Loai	Size	Mau	GiaNhap	GiaBan	Ton	DaBan
1	Aothun basic	Aothun	M	Den	80000	150000	50	0
102	Ao croptop	Aothun	S	Trang	90000	170000	40	0
201	Somi slimfit	Aosomi	L	Xanh	200000	350000	30	0
202	Somi oxford	Aosomi	XL	Xam	220000	380000	25	0
301	Quanjean skinny	Quanjean	M	Den	300000	550000	20	0
302	Quankaki classic	Quankaki	L	Nau	250000	450000	30	0
402	Tatcongan	Phukien	S	Trang	10000	15000	200	0

Hình 7.3: Danh sách sản phẩm sau khi đã xóa thành công mã 401

II. Quản lý kho

1. Mục tiêu và ý nghĩa .

Module Quản lý kho hàng là một phân hệ mở rộng kết nối trực tiếp với nền tảng Quản lý sản phẩm. Chức năng không chỉ dừng lại ở việc lưu trữ mà còn tập trung vào tối ưu hóa nghiệp vụ kiểm soát số lượng hàng hóa. Thông qua các thuật toán duyệt và xử lý trên Danh sách liên kết đơn, hệ thống cung cấp các công cụ trực quan giúp người quản trị nắm bắt chính xác dòng chảy hàng hóa, đưa ra cảnh báo kịp thời về tình trạng thiếu hụt, từ đó đảm bảo dữ liệu luôn được chuẩn hóa, nhất quán và an toàn thông qua cơ chế lưu trữ tệp tin.

2. Cấu trúc dữ liệu

- **Cấu trúc (struct):** Phân hệ sử dụng cấu trúc struct node để đóng gói dữ liệu sản phẩm (sanpham data) và quản lý bằng cơ chế bộ nhớ động của **Danh sách liên kết đơn (Linked List)**. Việc này giúp hệ thống xử lý số lượng sản phẩm linh hoạt, không bị giới hạn bộ nhớ như mảng tĩnh.
- **Con trỏ toàn cục head:** Luôn giữ địa chỉ của nút đầu tiên trong danh sách liên kết. Đây là điểm truy cập duy nhất để phân hệ thực hiện mọi thao tác duyệt tuyến tính (như tính tổng tồn kho, tìm sản phẩm cực trị).
- **Cơ chế lưu trữ bền vững:** Mọi biến động số lượng vật lý trong kho sau khi cập nhật đều được đồng bộ trực tiếp xuống tệp văn bản sanpham.txt thông qua hàm luusanpham(), giúp dữ liệu tồn tại vĩnh viễn sau khi đóng chương trình.

3. Phân tích các hàm chức năng trọng tâm

3.1. Hàm nhập kho (nhapkho)

```
void nhapkho(){
    int ma;
    int sl;
    printf("\n===== NHAP KHO =====\n");
    printf("Nhap ma san pham: ");
    scanf("%d",&ma);
    node *p = timtheoma(ma);
    if(p == NULL){
        printf("Khong tim thay san pham!\n");
        return;
    }
    printf("\nSan pham: %s\n", p->data.tensp);
    printf("Ton hien tai: %d\n", p->data.soluong);
    printf("Nhap so luong them: ");
    scanf("%d",&sl);
    if(sl <= 0){
        printf("So luong khong hop le!\n");
        return;
    }
    p->data.soluong += sl;
    luusanpham();
    printf("\nNhap kho thanh cong!\n");
    printf("Ton moi: %d\n",p->data.soluong);
}
```

Hình 3.1: : Giao diện thực thi chức năng Nhập kho trên màn hình Console

3.1.1. Mục đích

Cho phép cộng thêm số lượng hàng hóa thực tế khi có đợt hàng mới về. Hàm đóng vai trò cập nhật động dữ liệu kinh doanh mà không làm thay đổi các thuộc tính định danh gốc của sản phẩm.

3.1.2. Các điểm kỹ thuật trọng tâm

- **Xác thực thông tin (Validation):** Hàm sử dụng module hỗ trợ timtheoma(ma) để kiểm tra sự tồn tại của hàng hóa trước khi xử lý. Nếu con trỏ trả về NULL (không tìm thấy), thao tác lập tức bị chặn bằng lệnh return để tránh xung đột hoặc ghi đè rác dữ liệu.
- **Kiểm soát tính hợp lệ đầu vào:** Đưa vào điều kiện kiểm tra logic $sl \leq 0$. Cơ chế này loại bỏ các trường hợp nhập sai số lượng âm hoặc bằng không từ bàn phím.
- **Đồng bộ dữ liệu trực thời (Synchronization):** Sau khi thực hiện phép toán tích lũy số lượng $p->data.soluong += sl$, hàm gọi ngay

luusanpham() để đồng bộ hóa từ bộ nhớ RAM xuống đĩa cứng, đảm bảo an toàn thông tin tối đa.

3.1.3. Quy trình xử lý (Workflow)

1. **Tiếp nhận định danh:** Người dùng nhập mã sản phẩm cần nhập kho.
2. **Kiểm tra & Định vị:** Gọi hàm timtheoma() để tìm kiếm nút tương ứng. Nếu không tồn tại, báo lỗi. Nếu tồn tại, hiển thị thông tin tên sản phẩm (tensp) và số lượng tồn hiện tại.
3. **Nhập liệu chi tiết:** Yêu cầu người dùng nhập số lượng bổ sung và xác thực điều kiện $S > 0$.
4. **Cập nhật & Ghi tệp:** Thay đổi thuộc tính số lượng tại nút và gọi hàm luusanpham() để lưu trữ bền vững.

3.1.4. Bảng minh họa sản phẩm mẫu

```
=====
                        QUAN LY KHO
=====
1. Nhập kho
2. Kiểm tra tồn kho
3. Cảnh báo hết hàng
4. Tổng hàng tồn kho
5. Tồn kho nhiều nhất
6. Tồn kho ít nhất
0. Quay lại
=====
Nhập lựa chọn: 1
===== NHAP KHO =====
Nhập mã sản phẩm: 101
Sản phẩm: AO THUN NAM
Tồn hiện tại: 25
Nhập số lượng thêm: 15
Nhập kho thành công!
Tồn mới: 40
```

3.2. Hàm kiểm tra tồn kho (kiemtratonkho)

```
void kiemtratonkho(){
    if(head == NULL){
        printf("\nKhong co du lieu!\n");
        return;
    }
    printf("\n===== TON KHO =====\n");
    printf("%-8s%-30s%-15s\n",
        "Ma",
        "Ten san pham",
        "So luong");
    printf("===== \n");
    node *p = head;
    while(p != NULL){
        printf("%-8d%-30s%-15d\n", p->data.masp, p->data.tensp, p->data.soluong);
        p = p->next;
    }
}
```


Hình 3.2 :*Giao diện thực thi chức năng Kiểm tra tồn kho trên màn hình Console*

3.2.1. Mục đích

Cung cấp cái nhìn trực quan và tổng quát cho người quản trị về trạng thái số lượng của toàn bộ các mặt hàng đang lưu trữ trong hệ thống.

3.2.2. Kỹ thuật lập trình trọng tâm

- **Xử lý ngoại lệ:** Chương trình luôn đặt điều kiện chặn if(head == NULL) lên đầu hàm. Đây là kỹ thuật xử lý an toàn giúp thông báo rõ ràng cho người dùng khi danh sách rỗng thay vì để màn hình lỗi hệ thống.
- **Định dạng bảng biểu (Tabular Formatting):** Sử dụng các đặc tả định dạng căn lề trái của hàm printf:
 - %-8s (Mã), %-30s (Tên sản phẩm), %-15s (Số lượng).
 - Dấu trừ (-) biểu thị việc căn lề trái, con số đi kèm quy định độ rộng cố định của cột. Kỹ thuật này giữ cho bảng dữ liệu luôn ngay ngắn, chuyên nghiệp dù chuỗi tên sản phẩm có độ dài ngắn khác nhau.
- **Duyệt tuyến tính:** Sử dụng con trỏ tạm node *p = head làm con trỏ chạy, tịnh tiến qua từng nút bằng lệnh p = p->next cho đến khi chạm NULL để xuất toàn bộ danh mục.

3.2.3. Bảng minh họa sản phẩm mẫu

```
=====
          QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hang ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
Nhap lua chon: 2
=====
===== TON KHO =====
Ma          Ten san pham          So luong
=====
101         AO THUN NAM                40
102         AO SO MI NAM           12
103         QUAN JEAN NAM          18
104         VAY NU                 8
105         AO KHOAC NAM           5
106         CHAN VAY NU            3
107         QUAN KAKI NAM          30
```

3.3. Hàm cảnh báo hết hàng (canhbaohethang)

```
void canhbaohethang(){
    node *p = head;
    int found = 0;
    printf("\n===== CANH BAO HET HANG =====\n");
    while(p != NULL){
        if(p->data.soluong <= 5){
            printf("\nMa SP      : %d", p->data.masp);
            printf("\nTen SP       : %s", p->data.tensp);
            printf("\nCon lai      : %d\n", p->data.soluong);
            found = 1;
        }
        p = p->next;
    }
    if(found == 0){
        printf("Tat ca san pham deu du hang.\n");
    }
}
```

Hình 3.3: Giao diện thực thi chức năng Cảnh báo hết hàng trên màn hình Console

3.3.1. Mục đích

Đóng vai trò là một tính năng tối ưu hóa trải nghiệm quản lý (UX), tự động lọc và phát hiện sớm các mặt hàng có nguy cơ đứt gãy chuỗi cung ứng (sắp hết hàng) để người quản trị kịp thời có kế hoạch nhập hàng mới.

3.3.2. Kỹ thuật lập trình trọng tâm

- **Thuật toán lọc dữ liệu:** Duyệt tuần tự qua toàn bộ danh sách liên kết từ đầu đến cuối. Tại mỗi nút, hệ thống kiểm tra điều kiện biên: $p \rightarrow \text{data.soluong} \leq 5$.
- **Cơ chế cờ hiệu (Flag Mechanism):** Sử dụng biến `int found = 0` làm cờ giám sát luồng hiển thị giao diện. Nếu có ít nhất một sản phẩm thỏa mãn điều kiện, cờ sẽ bật lên `found = 1`. Nếu duyệt hết danh sách mà `found` vẫn giữ nguyên giá trị 0, hệ thống sẽ xuất ra thông báo khẳng định sự an toàn của kho hàng, giúp luồng phản hồi thông tin luôn minh bạch và tường minh.

3.3.3. Bảng minh họa sản phẩm mẫu

```
=====
                        QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hanc ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
Nhap lua chon: 3

===== CANH BAO HET HANG =====
Ma SP      : 105
Ten SP     : AO KHOAC NAM
Con lai    : 5

Ma SP      : 106
Ten SP     : CHAN VAY NU
Con lai    : 3
```

3.4. Hàm thống kê tổng tồn kho (thongketongtonkho)

```
void thongketongtonkho(){
    int tong = 0;
    node *p = head;
    while(p != NULL){
        tong += p->data.soluong;
        p = p->next;
    }
    printf("\nTong so luong hang ton kho: %d\n", tong);
}
```

Hình 3.4: Giao diện thực thi chức năng Thống kê tổng tồn kho trên màn hình Console

3.4.1. Mục đích

Thực hiện chức năng tính toán tổng lượng vốn bằng hàng hóa đang bị tồn đọng trong kho, cung cấp số liệu tổng quan phục vụ cho các phép toán thống kê tài chính cao hơn.

3.4.2. Kỹ thuật lập trình trọng tâm

- **Kỹ thuật tích lũy dữ liệu:** Khởi tạo một biến tích lũy `int tong = 0`.
- **Vòng lặp duyệt phẳng:** Con trỏ `p` di chuyển tuyến tính qua từng nút dữ liệu. Tại mỗi điểm dừng, chương trình lấy giá trị số lượng của sản phẩm đó cộng dồn trực tiếp vào biến lưu trữ: `tong += p->data.soluong`; Thuật toán đạt độ phức tạp thời gian là $\mathcal{O}(n)$, tối ưu cho cấu trúc danh sách liên kết đơn.

3.4.3. . Bảng minh họa sản phẩm mẫu

```

=====
QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hang ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
hap lua chon: 4

Tong so luong hang ton kho: 116
=====
```

3.5. Hàm tìm sản phẩm tồn kho nhiều nhất / ít nhất (tonkhonhieunhat / tonkhoitnhat)

```

void tonkhonhieunhat(){
    if(head == NULL){
        printf("Danh sach rong!\n");
        return;
    }
    node *max = head;
    node *p = head->next;
    while(p != NULL){
        if(p->data.soluong > max->data.soluong){
            max = p;
        }
        p = p->next;
    }
    printf("\n===== SAN PHAM TON KHO NHIEU NHAT =====\n");
    printf("Ma SP      : %d\n", max->data.masp);
    printf("Ten SP      : %s\n", max->data.tensp);
    printf("So luong    : %d\n", max->data.soluong);
}

```

Hình 3.5.1: Giao diện thực thi chức năng tìm sản phẩm tồn kho nhiều nhất trên màn hình Console

```

void tonkhoitnhat(){
    if(head == NULL){
        printf("Danh sach rong!\n");
        return;
    }
    node *min = head;
    node *p = head->next;
    while(p != NULL){
        if(p->data.soluong < min->data.soluong){
            min = p;
        }
        p = p->next;
    }
    printf("\n===== SAN PHAM TON KHO IT NHAT =====\n");
    printf("Ma SP      : %d\n", min->data.masp);
    printf("Ten SP      : %s\n", min->data.tensp);
    printf("So luong    : %d\n", min->data.soluong);
}

```

Hình 3.5.2: Giao diện thực thi chức năng tìm sản phẩm tồn kho ít nhất trên màn hình Console

3.5.1. Mục đích

Giúp người quản lý định vị nhanh chóng hai thái cực của hàng hóa: Mặt hàng nào đang tồn đọng quá nhiều (cần xả kho, kích cầu) và mặt hàng nào đang ít nhất (cần ưu tiên tái đặt hàng).

3.5.2. Kỹ thuật lập trình trọng tâm (Thuật toán tìm cực trị)

- **Khởi tạo giả định:** Sau khi vượt qua bước kiểm tra danh sách rỗng, thuật toán thiết lập giả định phần tử đầu tiên (head) giữ giá trị cực trị (max hoặc min).
- **Vòng lặp tối ưu:** Con trỏ p bắt đầu duyệt từ phần tử kế tiếp (head->next). Điều này giúp tiết kiệm 1 bước so sánh không cần thiết.
- **Cập nhật con trỏ địa chỉ:** Thay vì tốn bộ nhớ để sao chép lại toàn bộ dữ liệu của sản phẩm đạt cực trị, thuật toán chỉ thực hiện lưu lại địa chỉ của nút chứa sản phẩm đó (max = p hoặc min = p). Kỹ thuật con trỏ này giúp tối ưu hóa hiệu năng hệ thống và tốc độ xử lý của RAM.

3.5.3. Bảng minh họa sản phẩm mẫu

```
=====
                     QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hang ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
Nhap lua chon: 5.

===== SAN PHAM TON KHO NHIEU NHAT =====
Ma SP      : 101
Ten SP     : AO THUN NAM
So luong   : 40
```

```

=====
QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hang ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
Nhap lua chon: 6.

===== SAN PHAM TON KHO IT NHAT =====
Ma SP      : 106
Ten SP     : CHAN VAY NU
So luong   : 3

```

3.6. Hàm điều hướng Menu Kho (menukho)

```

void menukho(){
    int chon;
    do{
        printf("\n");
        printf("=====\\n");
        printf("        QUAN LY KHO\\n");
        printf("=====\\n");
        printf("1. Nhap kho\\n");
        printf("2. Kiem tra ton kho\\n");
        printf("3. Canh bao het hang\\n");
        printf("4. Tong hang ton kho\\n");
        printf("5. Ton kho nhieu nhat\\n");
        printf("6. Ton kho it nhat\\n");
        printf("0. Quay lai\\n");
        printf("=====\\n");
        printf("Nhap lua chon: ");
        scanf("%d",&chon);
        switch(chon){
            case 1:
                nhapkho();
                break;
            case 2:
                kiemtratonkho();
                break;
            case 3:
                canhbaohethang();
                break;
            case 4:
                thongketongtonkho();
                break;
            case 5:
                tonkhonhieunhat();
                break;
            case 6:
                tonkhoitnhat();
                break;
            case 0:
                break;
            default:
                printf("Lua chon khong hop le!\\n");
        }
    } while(chon != 0);
}

```

Hình 3.6: Giao diện thực thi chức năng điều hướng Menu Kho trên màn hình Console

3.6.1. Mục đích

Xây dựng giao diện tương tác phân cấp (Console Interface), giúp người quản trị dễ dàng định hướng, lựa chọn và thực hiện nhanh các tác vụ quản lý kho.

3.6.2. Kỹ thuật lập trình trọng tâm

- **Vòng lặp kiểm soát trạng thái (do...while):** Đảm bảo giao diện menu luôn được tái hiển thị sau khi một hàm chức năng kết thúc, duy trì phiên làm việc cho đến khi người dùng chủ động chọn lệnh thoát (chon != 0).
- **Cấu trúc điều hướng rẽ nhánh (switch...case):** Tiếp nhận giá trị số nguyên từ bàn phím để ánh xạ trực tiếp đến địa chỉ vùng nhớ của hàm chức năng tương ứng, đảm bảo tính đầy đủ và rành mạch trong kiến trúc mã nguồn.

3.6.3. . Bảng minh họa sản phẩm mẫu

```
=====
                        QUAN LY KHO
=====
1. Nhap kho
2. Kiem tra ton kho
3. Canh bao het hang
4. Tong hang ton kho
5. Ton kho nhieu nhat
6. Ton kho it nhat
0. Quay lai
=====
Nhap lua chon:
```

III. QUẢN LÝ BÁN HÀNG

1. Mục tiêu và ý nghĩa

- Mục tiêu tối thượng của module này là **tối ưu hóa toàn bộ quy trình từ khi tiếp cận khách hàng cho đến khi dòng tiền đổ về tài khoản doanh nghiệp**. Cụ thể bao gồm:
 - **Tự động hóa quy trình (Automation):** Chuẩn hóa và tự động hóa các bước từ báo giá, lên đơn hàng (SO), kiểm tra tồn kho, xuất kho cho đến lập hóa đơn. Giảm thiểu tối đa việc nhập liệu thủ công và sai sót con người.
 - **Quản lý thông tin tập trung:** Lưu trữ toàn bộ dữ liệu về khách hàng, lịch sử mua hàng, chính sách giá, chiết khấu và công nợ trên một nền tảng duy nhất.
 - **Kiểm soát tồn kho và cung ứng kịp thời:** Kết nối chặt chẽ với kho hàng để biết chính xác lượng hàng khả dụng, tránh tình trạng hứa hẹn với khách nhưng hết hàng (overselling) hoặc om hàng quá lâu.
 - **Tối ưu hóa doanh thu và dòng tiền:** Theo dõi sát sao các khoản phải thu (công nợ khách hàng), nhắc nhở thanh toán đúng hạn để đảm bảo vòng quay vốn cho doanh nghiệp.
 - **Cung cấp bức tranh dữ liệu trực quan:** Giúp ban lãnh đạo theo dõi doanh số theo thời gian thực (real-time), dự báo xu hướng thị trường và đánh giá hiệu suất của từng nhân viên kinh doanh.

2. Cấu trúc dữ liệu

2.1. Tóm tắt quy trình chức năng bán hàng

Chức năng bán hàng được xây dựng nhằm hỗ trợ nhân viên thực hiện giao dịch bán sản phẩm cho khách hàng một cách nhanh chóng và chính xác. Quy trình hoạt động của chức năng này gồm các bước sau:

Bước 1: Nhân viên nhập mã sản phẩm cần bán.

Bước 2: Hệ thống tìm kiếm sản phẩm theo mã đã nhập và kiểm tra sản phẩm có tồn tại trong danh sách hay không.

Bước 3: Nếu sản phẩm tồn tại, hệ thống hiển thị thông tin sản phẩm bao gồm tên sản phẩm, giá bán và số lượng tồn kho hiện tại.

Bước 4: Nhân viên nhập số lượng sản phẩm khách hàng muốn mua.

Bước 5: Hệ thống kiểm tra tính hợp lệ của số lượng nhập vào và đối chiếu với số lượng tồn kho hiện có.

Bước 6: Nếu đủ hàng, hệ thống cập nhật lại số lượng tồn kho và số lượng sản phẩm đã bán.

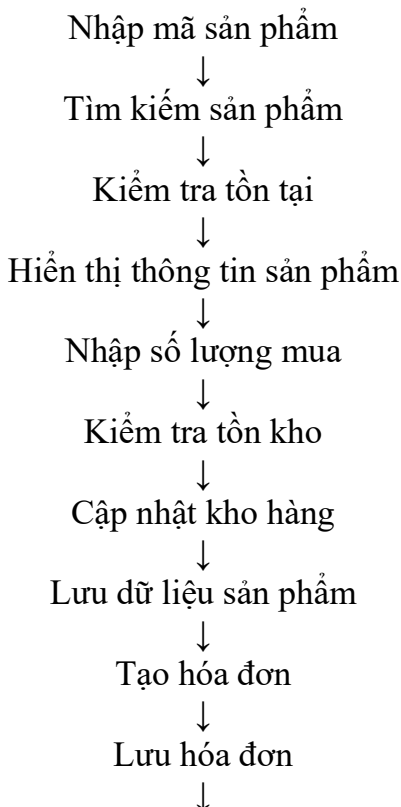
Bước 7: Dữ liệu sản phẩm sau khi cập nhật được lưu xuống file để đảm bảo tính toàn vẹn của dữ liệu.

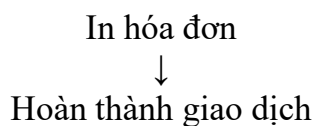
Bước 8: Hệ thống tiến hành tạo hóa đơn bán hàng, tự động sinh mã hóa đơn và tính thành tiền dựa trên số lượng mua và đơn giá sản phẩm.

Bước 9: Hóa đơn được lưu vào danh sách hóa đơn và ghi xuống file để phục vụ cho việc quản lý, tra cứu và thống kê doanh thu.

Bước 10: Hệ thống hiển thị hóa đơn và thông báo giao dịch bán hàng thành công.

Sơ đồ quy trình:





2.2 Cấu trúc hóa đơn

```
printf("\n===== BAN HANG =====\n");  
printf("Nhap ma san pham: ");  
scanf("%d", &ma);
```

```
node *sp = timtheoma(ma);
```

```
if(sp == NULL)  
{  
    printf("Khong tim thay san pham!\n");  
    return;  
}
```

Giải thích:

Đầu tiên chương trình hiển thị giao diện chức năng bán hàng và yêu cầu người dùng nhập mã sản phẩm. Sau đó hàm `timtheoma(ma)` được gọi để tìm kiếm sản phẩm tương ứng trong danh sách liên kết. Kết quả tìm kiếm được lưu vào con trỏ `sp`. Nếu không tìm thấy sản phẩm, con trỏ sẽ nhận giá trị `NULL`, chương trình thông báo lỗi và kết thúc chức năng bán hàng.

3. Phân tích các hàm chức năng trọng tâm

3.1 Hiển thị thông tin sản phẩm

```
printf("\nTen san pham : %s\n", sp->data.tensp);  
printf("Gia ban      : %.0f\n", sp->data.giaban);  
printf("Ton kho      : %d\n", sp->data.soluong);
```

Giải thích:

Sau khi tìm thấy sản phẩm, chương trình hiển thị tên sản phẩm, giá bán và số lượng tồn kho hiện tại. Việc hiển thị này giúp người bán kiểm tra chính xác mặt hàng trước khi tiến hành giao dịch với khách hàng.

3.2. Nhập và kiểm tra số lượng mua

```
printf("\nNhap so luong mua: ");  
scanf("%d", &sl);
```

```
if(sl <= 0)  
{
```

```

    printf("So luong khong hop le!\n");
    return;
}

if(sl > sp->data.soluong)
{
    printf("Khong du hang trong kho!\n");
    return;
}

```

Giải thích:

Người dùng nhập số lượng sản phẩm khách muốn mua và giá trị được lưu vào biến sl. Chương trình kiểm tra số lượng nhập vào có hợp lệ hay không. Nếu số lượng nhỏ hơn hoặc bằng 0 thì giao dịch bị từ chối. Tiếp theo, chương trình kiểm tra lượng hàng tồn kho. Nếu số lượng khách mua lớn hơn số lượng hiện có trong kho thì hệ thống sẽ thông báo không đủ hàng.

3.3. Cập nhật dữ liệu kho hàng

```

sp->data.soluong -= sl;
sp->data.daban += sl;

```

```

luusanpham();

```

Giải thích:

Khi giao dịch hợp lệ, chương trình tiến hành cập nhật dữ liệu sản phẩm. Số lượng tồn kho được giảm đi tương ứng với số lượng đã bán, đồng thời số lượng đã bán được cộng thêm để phục vụ cho việc thống kê sau này. Sau đó hàm luusanpham() được gọi để lưu lại dữ liệu mới vào file.

3.4. Tạo hóa đơn bán hàng

```

void taohoadon(node *sp, int soluong)
{
    hoadon hd;

    hd.mahd = mahoadontudong++;
    hd.masp = sp->data.masp;

    strcpy(hd.tensp, sp->data.tensp);

    hd.soluong = soluong;
    hd.dongia = sp->data.giaban;
}

```

```

        hd.thanhtien = hd.soluong * hd.dongia;
    }

```

Giải thích:

Hàm taohoadon() được sử dụng để tạo một hóa đơn mới. Chương trình tự động sinh mã hóa đơn, lưu mã sản phẩm, tên sản phẩm, số lượng mua và đơn giá bán. Thành tiền được tính bằng cách lấy số lượng nhân với đơn giá.

3.5. Lưu hóa đơn vào hệ thống

```

dshoadon[soloadon++] = hd;

```

```

luuhoadon();

```

Giải thích:

Sau khi tạo thành công, hóa đơn được lưu vào mảng dshoadon. Biến sooloadon được tăng lên để quản lý số lượng hóa đơn hiện có. Tiếp theo, hàm luuhoadon() được gọi để ghi dữ liệu hóa đơn xuống file hoadon.txt.

3.6. In hóa đơn cho khách hàng

```

printf("\n=====\\n");
printf("        HOA DON\\n");
printf("=====\\n");
printf("Ma hoa don : %d\\n", hd.mahd);
printf("Ma SP    : %d\\n", hd.masp);
printf("Ten SP    : %s\\n", hd.tensp);
printf("So luong  : %d\\n", hd.soluong);
printf("Don gia   : %.0f\\n", hd.dongia);
printf("Thanh tien : %.0f\\n", hd.thanhtien);
printf("=====\\n");

```

Giải thích:

Sau khi hóa đơn được tạo thành công, chương trình hiển thị toàn bộ thông tin giao dịch bao gồm mã hóa đơn, mã sản phẩm, tên sản phẩm, số lượng mua, đơn giá và thành tiền. Đây là bước xác nhận cuối cùng của quá trình bán hàng.

3.7. Thông báo hoàn tất giao dịch

```

taohoadon(sp, sl);

```

```

printf("\nBan hang thanh cong!\\n");

```

Giải thích:

Sau khi dữ liệu kho hàng được cập nhật và hóa đơn được tạo thành công, chương trình hiển thị thông báo bán hàng thành công. Điều này cho biết giao dịch đã được xử lý hoàn chỉnh và lưu vào hệ thống.

Cách chia này rất giống cách viết trong **Chương 3: Phân tích và thiết kế chương trình** hoặc **Mô tả thuật toán chức năng bán hàng** của báo cáo đề án

3. 8. Chạy thử phần mềm

3.8.1 Các bước chạy thử

Bước 1: Từ menu chính chọn

```
=====
      QUAN LY SHOP THOI TRANG
=====
1. Quan ly san pham
2. Quan ly kho
3. Ban hang
4. Thong ke
0. Thoat
=====
```

Chon:

Nhap:

3
đề vào menu bán hàng.

Bước 2: Chọn chức năng Bán hàng

Menu hiện:

```
=====
      QUAN LY BAN HANG
=====
1. Ban hang
```

2. Danh sach hoa don
3. Tim hoa don
4. Tong doanh thu
5. Hoa don lon nhat
0. Quay lai

=====

Nhap lua chon:

Nhap:

1

Bước 3: Nhập mã sản phẩm

Chương trình chạy hàm:

void banhang()

và hỏi:

===== BAN HANG =====

Nhap ma san pham:

Ví dụ nhập:

101

Bước 4: Chương trình hiển thị thông tin sản phẩm

Nếu mã 101 tồn tại:

Ten san pham : Ao Thun

Gia ban : 250000

Ton kho : 20

Bước 5: Nhập số lượng khách mua

Nhap so luong mua:

Ví dụ:

3

Bước 6: Hệ thống xử lý

Đoạn code:

sp->data.soluong -= sl;

sp->data.daban += sl;

sẽ:

- Trừ tồn kho:

20 -> 17

- Tăng số lượng đã bán:

0 -> 3

Sau đó gọi:

taohoadon(sp, sl);

để tạo hóa đơn.

Bước 7: Hóa đơn được in ra

Ví dụ:

```
=====
HOA DON
=====
Ma hoa don : 1
Ma SP      : 101
Ten SP     : Ao Thun
So luong   : 3
Don gia    : 250000
Thanh tien : 750000
=====
```

Trong đó:

Thanh tien = So luong * Don gia

= 3 * 250000

= 750000

Bước 8: Thông báo thành công

Ban hang thanh cong!

Kiểm tra hóa đơn vừa tạo

Từ menu bán hàng chọn:

2

Danh sách hóa đơn:

```
=====
=====
MaHD   MaSP   TenSP           SL   DonGia   ThanhTien
=====
1      101    Ao Thun          3    250000   750000
=====
```

Kiểm tra doanh thu

Chọn:

4

Kết quả:

Tổng doanh thu hiện tại: 750000 VND

Chạy thử hoàn chỉnh

Giả sử trước đó bạn đã thêm sản phẩm:

Mã	Tên SP	Giá bán	Tồn kho
101	AoThun	250000	20

01

Thao tác nhập:

3

1

101

3

2

4

0

Ý nghĩa:

3 -> Vào quản lý bán hàng

1 -> Bán hàng

101 -> Mã sản phẩm

3 -> Mua 3 sản phẩm

2 -> Xem danh sách hóa đơn

4 -> Xem doanh thu

0 -> Quay lại

Kết quả cuối cùng:

Tồn kho còn: 17

Đã bán: 3

Doanh thu: 750000

Số hóa đơn: 1

Lưu ý: Với code hiện tại, trước khi bán hàng bạn phải vào **Quản lý sản phẩm** → **Thêm sản phẩm** để có dữ liệu, nếu không nhập mã sản phẩm sẽ báo:

Không tìm thấy sản phẩm!

3.8.2 Mục đích:

Phần Quản lý bán hàng được xây dựng nhằm hỗ trợ cửa hàng thực hiện việc bán sản phẩm cho khách hàng, đồng thời tự động cập nhật tồn kho và lưu thông tin hóa đơn sau mỗi lần bán.

Chức năng này giúp:

- Quản lý quá trình bán hàng.
- Theo dõi số lượng sản phẩm đã bán.
- Cập nhật số lượng tồn kho.
- Lưu trữ hóa đơn.
- Thống kê doanh thu của cửa hàng.

IV. Thống Kê

1. Mục tiêu và ý nghĩa

Module Thống kê là phân hệ hỗ trợ phân tích dữ liệu kinh doanh trong hệ thống quản lý shop thời trang. Chức năng có nhiệm vụ tổng hợp và xử lý dữ liệu từ danh sách sản phẩm và hóa đơn nhằm cung cấp các thông tin quan trọng như doanh thu, lợi nhuận, sản phẩm bán chạy, sản phẩm bán chậm và tình trạng hàng hóa. Thông qua các thuật toán duyệt danh sách liên kết đơn và xử lý mảng dữ liệu, hệ thống giúp người quản lý đưa ra các quyết định kinh doanh chính xác và hiệu quả hơn.

2. Cấu trúc dữ liệu

• Cấu trúc sản phẩm (sanpham):

Lưu trữ thông tin sản phẩm gồm mã sản phẩm, tên sản phẩm, loại, kích thước, màu sắc, giá nhập, giá bán, số lượng tồn kho và số lượng đã bán.

• Danh sách liên kết đơn (Linked List):

Mỗi node chứa một sản phẩm và con trỏ liên kết đến node kế tiếp. Danh sách được quản lý thông qua con trỏ đầu danh sách head.

- **Cấu trúc hóa đơn (hoadon):**

Lưu trữ thông tin giao dịch bán hàng gồm mã hóa đơn, mã sản phẩm, số lượng mua, đơn giá và thành tiền.

- **Cơ chế lưu trữ dữ liệu:**

Thông tin sản phẩm được lưu trong tệp sanpham.txt và thông tin hóa đơn được lưu trong tệp hoadon.txt nhằm đảm bảo dữ liệu được bảo toàn sau khi kết thúc chương trình.

3. Phân tích các hàm chức năng trọng tâm

3.1. Hàm thống kê doanh thu

3.1.1. Mục đích

Tính tổng doanh thu từ tất cả các hóa đơn đã được lập trong hệ thống.

3.1.2. Kỹ thuật lập trình trọng tâm

- Sử dụng biến tích lũy doanhthu để cộng dồn giá trị thanhtien của từng hóa đơn.
- Duyệt tuần tự mảng dshoadon[[]].
- Độ phức tạp thuật toán $O(n)$.

```
void thongkedoanhthu() {  
    float doanhthu = 0;  
  
    for(int i = 0; i < sohoadon; i++) {  
        doanhthu += dshoadon[i].thanhtien;  
    }  
  
    printf("\nTong doanh thu: %.0f VND", doanhthu);  
}
```

Hình 3.1.2: : *Giao diện thực thi chức năng Doanh thu trên màn hình Console*

- B1. Khởi tạo doanh thu = 0.
- B2. Duyệt toàn bộ mảng hóa đơn.
- B3. Cộng dồn trường thanh tiền.
- B4. Xuất tổng doanh thu ra màn hình.

3.1.4. Bảng minh họa sản phẩm mẫu

```
===== THONG KE DOANH THU =====  
  
Tong doanh thu: 4410000 VND  
  
=====
```

Hình 3.1: Giao diện thực thi chức năng Thống kê doanh thu trên màn hình Console.

3.2. Hàm thống kê lợi nhuận

3.2.1. Mục đích

Tính tổng lợi nhuận thu được từ hoạt động bán hàng.

3.2.2. Kỹ thuật lập trình trọng tâm

- Duyệt danh sách liên kết sản phẩm.
- Áp dụng công thức:

$$\text{Lợi nhuận} = (\text{Giá bán} - \text{Giá nhập}) \times \text{Đã bán}$$

- Cộng dồn lợi nhuận của từng sản phẩm.

```

void thongkeloinhuan() {
    node *p = head;
    float loinhuan = 0;

    while(p != NULL) {
        loinhuan += (p->data.giabán -
                    p->data.gianháp)
                    * p->data.dabán;
        p = p->next;
    }

    printf("\nTong loi nhuan: %.0f VND", loinhuan);
}

```

Hình 3.2.2: : *Giao diện thực thi chức năng Tính tổng lợi nhuận trên màn hình Console*

3.2.3. Quy trình xử lý

- B1. Khởi tạo biến tổng lợi nhuận.
- B2. Duyệt danh sách sản phẩm.
- B3. Tính lợi nhuận từng sản phẩm.
- B4. Cộng dồn vào tổng lợi nhuận.
- B5. Hiển thị kết quả.

3.3. Hàm tìm sản phẩm bán chạy nhất

3.3.1. Mục đích

Xác định sản phẩm có số lượng bán ra lớn nhất trong hệ thống.

```

void sanphambanchaynhat() {
    if(head == NULL) return;

    node *max = head;
    node *p = head->next;

    while(p != NULL) {
        if(p->data.daban >
            max->data.daban)
            max = p;

        p = p->next;
    }

    printf("\nSP ban chay nhat: %s",
        max->data.tensp);
}

```



Hình 3.3.1: : *Giao diện thực thi chức năng xác định sản phẩm bán chạy trên màn hình Console*

3.3.2. Kỹ thuật lập trình trọng tâm

- Thuật toán tìm cực đại (Maximum).
- Khởi tạo sản phẩm đầu tiên làm giá trị lớn nhất.
- Duyệt toàn bộ danh sách và cập nhật khi tìm thấy giá trị daban lớn hơn.

3.3.3. Quy trình xử lý

- B1.Kiểm tra danh sách rỗng.
- B2.Gán node đầu tiên làm sản phẩm bán chạy nhất.
- B3.Duyệt các node còn lại.
- B4.So sánh trường daban.
- B5.Xuất thông tin sản phẩm.

3.4. Hàm tìm sản phẩm bán ít nhất

3.4.1. Mục đích

Xác định sản phẩm có số lượng bán ra thấp nhất.

```
void sanphambanitnhat() {  
    if(head == NULL) return;  
  
    node *min = head;  
    node *p = head->next;  
  
    while(p != NULL) {  
        if(p->data.daban <  
            min->data.daban)  
            min = p;  
  
        p = p->next;  
    }  
  
    printf("\nSP bán ít nhất: %s",  
        min->data.tensp);  
}
```

Hình 3.4.1: : *Giao diện thực thi chức năng xác định sản phẩm bán ít nhất trên màn hình Console*

3.4.2. Kỹ thuật lập trình trọng tâm

- Thuật toán tìm cực tiểu (Minimum).
- Duyệt danh sách liên kết.
- Lưu địa chỉ node có giá trị daban nhỏ nhất.

3.4.3. Quy trình xử lý

- B1.Kiểm tra danh sách rỗng.
- B2.Gán node đầu tiên làm giá trị nhỏ nhất.
- B3.Duyệt danh sách.
- B4.Cập nhật khi tìm thấy daban nhỏ hơn.

5. Hiển thị kết quả.

3.4.4. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)

```
===== SAN PHAM BAN IT NHAT =====  
  
Ma SP : 104  
Ten SP: Ao khoac Bomber  
Da ban: 0  
  
=====
```

Hình 3.4: Giao diện thực thi chức năng Sản phẩm bán ít nhất trên màn hình Console.

3.5. Hàm sắp xếp theo giá bán

3.5.1. Mục đích

Sắp xếp danh sách sản phẩm theo giá bán tăng dần nhằm hỗ trợ tra cứu và phân tích dữ liệu.

```
void sapxeptheogia() {  
    node *p, *q;  
    sanpham temp;  
  
    for(p = head; p != NULL; p = p->next) {  
        for(q = p->next; q != NULL; q = q->next) {  
  
            if(p->data.giaban >  
               q->data.giaban) {  
  
                temp = p->data;  
                p->data = q->data;  
                q->data = temp;  
  
            }  
        }  
    }  
}
```

Hình 3.5.1: : *Giao diện thực thi chức năng sắp xếp danh sách sản phẩm trên màn hình Console*

3.5.2. Kỹ thuật lập trình trọng tâm

- Sử dụng thuật toán Bubble Sort.
- So sánh trường giá bán giữa các node.
- Hoán đổi dữ liệu sản phẩm thay vì thay đổi liên kết node.

3.5.3. Quy trình xử lý

- B1. Duyệt danh sách bằng hai vòng lặp.
- B2. So sánh giá bán giữa các sản phẩm.
- B3. Hoán đổi dữ liệu nếu không đúng thứ tự.
- B4. Hiển thị danh sách sau khi sắp xếp.

3.5.4. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)

```
===== SAP XEP THEO GIA BAN =====  
  
101  Ao thun Polo           250000  
102  Quan Jean Slimfit      380000  
103  Vay hoa nhi            450000  
104  Ao khoac Bomber        500000  
  
=====
```

Hình 3.5: Giao diện thực thi chức năng Sắp xếp theo giá bán trên màn hình Console.

3.6. Hàm điều hướng Menu Thống kê

3.6.1. Mục đích

Tạo giao diện cho phép người dùng lựa chọn các chức năng thống kê.


```

void menuthongke() {
    int chon;
    do {
        scanf("%d",&chon);

        switch(chon) {
            case 1: thongkedoanhthu(); break;
            case 2: thongkeloinhuan(); break;
            case 3: sanphambanchaynhat(); break;
            case 4: sanphambanitnhat(); break;
            case 5: sapxeptheogia(); break;
        }
    } while(chon != 0);
}

```

Hình 3.6.1: : *Giao diện thực thi chức năng menu thống kê trên màn hình Console*

3.6.2. Kỹ thuật lập trình trọng tâm

- Sử dụng vòng lặp do...while.
- Điều hướng chức năng bằng cấu trúc switch...case.
- Duy trì phiên làm việc cho đến khi người dùng chọn thoát.

3.6.3. Bảng minh họa sản phẩm mẫu (Dữ liệu kiểm thử)

```
===== MENU THONG KE =====

1. Thong ke doanh thu
2. Thong ke loi nhuan
3. SP ban chay nhat
4. SP ban it nhat
5. Sap xep theo gia ban
0. Thoat

Chon: 3

===== SAN PHAM BAN CHAY NHAT =====

Ma SP : 103
Ten SP: Vay hoa nhi
Da ban: 20

=====
```



Hình 3.6: Giao diện thực thi chức năng Menu thống kê trên màn hình Console.

3.6.3. Quy trình xử lý (Workflow)

- B1. Hiển thị danh sách các chức năng thống kê.
- B2. Người dùng nhập lựa chọn từ bàn phím.
- B3. Chương trình kiểm tra giá trị nhập.
- B4. Thực hiện hàm tương ứng thông qua switch...case.
- B5. Khi hàm kết thúc, menu được hiển thị lại.
- B6. Nếu người dùng nhập 0, vòng lặp dừng và thoát khỏi menu.

4. Kết luận:

Module Thống kê đóng vai trò quan trọng trong việc hỗ trợ người quản lý đánh giá hiệu quả hoạt động kinh doanh thông qua các chỉ số doanh thu, lợi nhuận và tình hình tiêu thụ sản phẩm. Việc áp dụng danh sách liên kết đơn và mảng hóa đơn giúp chương trình xử lý dữ liệu hiệu quả, dễ mở

rộng và phù hợp với mô hình quản lý shop thời trang xây dựng bằng ngôn ngữ C.

LỜI KẾT

Qua quá trình thực hiện bài tập lớn với đề tài **“Xây dựng ứng dụng quản lý bán hàng quần áo thời trang”**, nhóm đã có cơ hội vận dụng những kiến thức đã học về lập trình để xây dựng một chương trình quản lý đơn giản, hỗ trợ các chức năng cơ bản như quản lý sản phẩm, khách hàng, hóa đơn và thống kê doanh thu.

Bài tập lớn giúp nhóm hiểu rõ hơn về cách phân tích yêu cầu, thiết kế chương trình, xây dựng các cấu trúc dữ liệu và triển khai các chức năng trong một ứng dụng thực tế. Đồng thời, đây cũng là cơ hội để các thành viên rèn luyện kỹ năng làm việc nhóm, tìm kiếm tài liệu và giải quyết các vấn đề phát sinh trong quá trình lập trình.

Do thời gian thực hiện và kiến thức còn hạn chế nên chương trình vẫn còn một số thiếu sót và cần được hoàn thiện thêm trong tương lai. Nhóm rất mong nhận được những ý kiến đóng góp từ thầy/cô để có thể cải thiện và nâng cao chất lượng sản phẩm cũng như kiến thức chuyên môn của mình.

Nhóm xin chân thành cảm ơn thầy/cô đã hướng dẫn và tạo điều kiện để nhóm hoàn thành bài tập lớn này ạ.